Московский государственный университет
имени М. В. Ломоносова
Факультет вычислительной математики и кибернетики

Отчет по заданию №6

«Сборка многомодульных программ. Вычисление корней уравнений и определенных интегралов.»

Вариант 6 / 4 / 3

Выполнил:
студент 103 группы
Огай В. Л.

Преподаватель:
Кузьменкова Е. А.

Москва
2024

Содержание

1 Постановка задачи **2**

2 Математическое обоснование **3**

2.1 Отрезки для корней и описание методов 3

2.2 Погрешности 5

3 Результаты экспериментов **6**

4 Структура программы и спецификация функций **7**

5 Сборка программы (Make-файл) **9**

6 Отладка программы, тестирование функций **10**

7 Анализ допущенных ошибок **11**

Список цитируемой литературы **12**

1 Постановка задачи

С заданной точностью ε вычислить площадь плоской фигуры, ограниченной тремя кривыми:

- $f_1(x) = 0.6x + 3$
- $f_2(x) = (x-2)^3 - 1$
- $f_3(x) = \frac{3}{x}$

Для вычисления площади плоской фигуры необходимо реализовать численные методы, предусмотренные вариантом задания. В данном варианте – комбинированный метод (хорд и касательных) для определения корней уравнений и метод парабол (метод Симпсона) для вычисления определенного интеграла от функции на отрезке. Отрезок $[a, b]$, содержащий корень, должен быть найден вручную с необходимым математическим обоснованием. Площадь фигуры необходимо вычислить с точностью $\varepsilon = 0.001$. ε_1 – точность, с которой считаются корни, и ε_2 – погрешность вычисления определенного интеграла, также должны быть выбраны с соответствующим математическим обоснованием.

2 Математическое обоснование

2.1 Отрезки для корней и описание методов

В варианте 6 представленного задания задана область, изображенная на (Рис.1), площадь которой необходимо найти. Для этого требуется вычислить абсциссы точек пересечения кривых, которые являются решением уравнения $F(x) = f(x) - g(x) = 0$. Для поиска корней данного уравнения используется комбинированный метод (хорд и касательных).

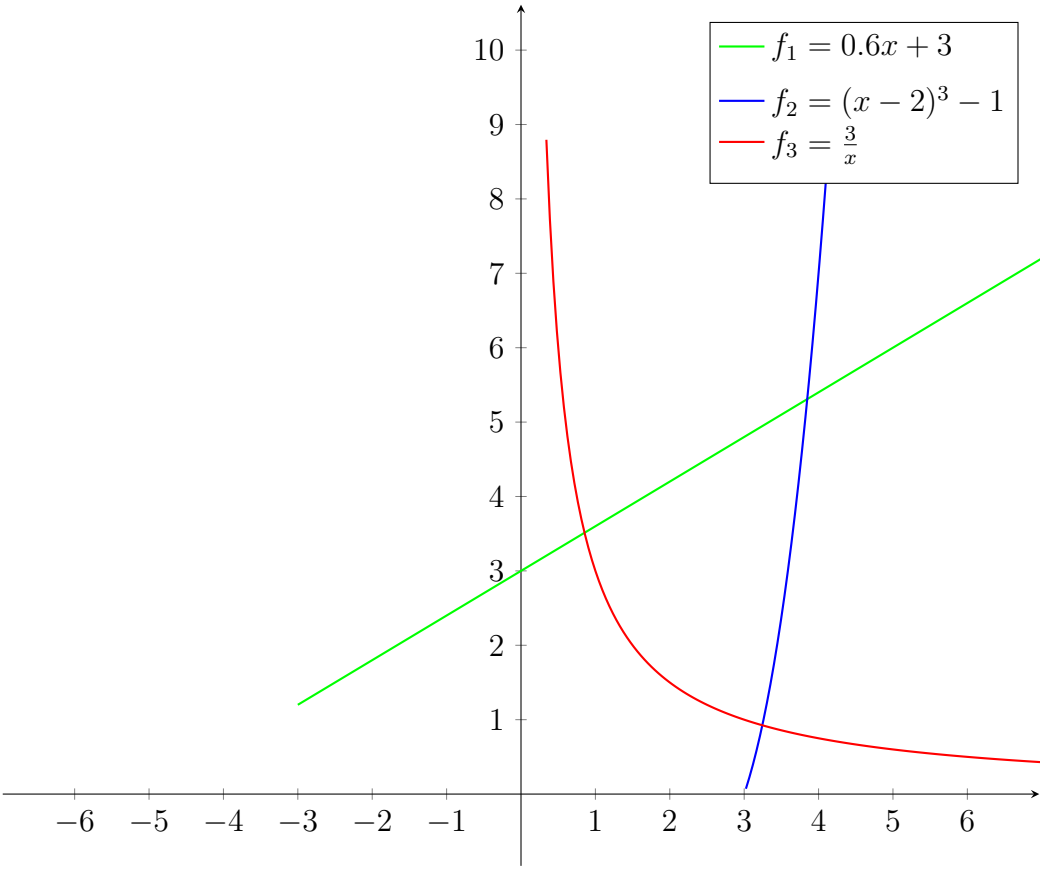

Рис. 1: Плоская фигура, ограниченная графиками заданных уравнений

Стоит различать два случая:
1. Первая и вторая производные функции $F(x)$ имеют одинаковый знак.
2. Первая и вторая производные функции $F(x)$ имеют разные знаки.

Комбинированный метод (хорд и касательных) – итеративный метод приближения корня уравнения $F(x) = 0$ на отрезке $[a, b]$. Требуется, чтобы функция была непрерывна на $[a, b]$, а вторая производная функции была монотонна на $[a, b]$. В случае 1 полагается, что

$$a_{n+1} = a_n - \frac{f(a_n)(b_n - a_n)}{f(b_n) - f(a_n)},\ b_{n+1} = b_n - \frac{f(b_n)}{f`(b_n)}.$$

В случае 2:

$$a_{n+1} = a_n - \frac{f(a_n)}{f'(a_n)},\ b_{n+1} = b_n - \frac{f(b_n)(b_n - a_n)}{f(b_n) - f(a_n)}.$$

Критерием остановки является условие $|b_{n+1} - a_{n+1}| < \varepsilon_1$. [1]

Здесь и далее

- $F_1(x) = f_1(x) - f_2(x)$;
- $F_2(x) = f_1(x) - f_3(x)$;
- $F_3(x) = f_2(x) - f_3(x)$.

Для поиска корней были выбраны отрезки: для $F_1(x)$ – $[2.5, 4]$, для $F_2(x)$ – $[0.5, 2.5]$, для $F_3(x)$ – $[2.5, 4]$. Проверим, что все функции удовлетворяют на нем необходимым условиям, а именно: на концах отрезка функция $F(x) = f(x) - g(x)$ принимает значения разных знаков, ее первые и вторые производные сохраняют знак.

1. $F_1(x) = 0.6x + 3 - ((x-2)^3 - 1)$.
   (a) $F_1(2.5) = 5.375 > 0, F_1(4) = -1.6 < 0$
   (b) $F_1'(x) = 0.6 - 3(x-2)^2 < 0\ \forall x \in [2.5, 4]$
   (c) $F_1''(x) = 6(x-2) > 0\ \forall x \in [2.5, 4]$
2. $F_2(x) = 0.6x + 3 - \frac{3}{x}$.
   (a) $F_2(0.5) = -2.7 < 0, F_2(2.5) = 3.3 > 0$
   (b) $F_2'(x) = 0.6 + \frac{3}{x^2} > 0\ \forall x \in [0.5, 2.5]$
   (c) $F_2''(x) = -\frac{6}{x^3} < 0\ \forall x \in [0.5, 2.5]$
3. $F_3(x) = (x-2)^3 - 1 - \frac{3}{x}$.
   (a) $F_3(2.5) = -2.075 < 0, F_1(4) = 6.25 > 0$
   (b) $F_3'(x) = 3(x-2)^2 + \frac{3}{x^2} > 0\ \forall x \in [2.5, 4]$
   (c) $F_3''(x) = 6(x-2) - \frac{6}{x^3} < 0\ \forall x \in [2.5, 4]$

Далее, после того как были найдены абсциссы точек пересечения кривых, необходимо вычислить определенные интегралы, используя метод парабол (метод Симпсона).

Метод парабол (метод Симпсона) позволяет приближённо вычислить определённый интеграл, прибегая к аппроксимации частей кривой многочленами второй степени.[2] На отрезке $[a, b]$ формула для определенного интеграла I следующая:

$$I_{2n} = \frac{h}{3}(f(a) + f(b)) + 4(f(x_1) + f(x_3) + ...f(x_{n-1})) + 2(f(x_2) + f(x_4)... + f(x_{n-2})),$$
где $x_i = a + ih$, $h = (b-a)/n$, n - чётно.

Критерий остановки в данном случае - правило Рунге:

$$|I - I_n| \approx \theta |I_{2n} - I_n|,$$

где θ равно $\frac{1}{15}$ для данного метода. Критерием остановки является условие $\theta|I_n - I_{2n}| < \varepsilon_2$. В таком случае метод возвращает I_{2n} в качестве ответа. [1]

2.2 Погрешности

Аддитивность абсолютных погрешностей при суммировании измеряемых результатов известна. Следовательно, ошибка при вычислении интегралов будет равна $3\varepsilon_2$. Выберем ε_1. Интегрируя функцию F в пределах x_1, x_2, которые были получены функцией *root*, получаем, что ошибка при вычислении $\leq$ суммы $\varepsilon_1 * max(F(x_1 \pm \varepsilon_1))$ и $\varepsilon_1 * max(F(x_2 \pm \varepsilon_1))$ – площадей прямоугольников (это значит, что избыточные или недостающие криволинейные трапеции будут лежать внутри этих прямоугольников). Нетрудно заметить, что на выбранном участке график f_1 лежит не ниже остальных графиков, следовательно погрешностями при интегрировании f_1 можно оценить сверху погрешности при интегрировании остальных функций. В силу достаточной малости ошибочного интервала и непрерывности f_1, любое используемое значение функции f_1 можно оценить сверху константой a такой, что прямая $y = a$ будет лежать сильно выше рассматриваемой фигуры. В данном случае, как видно из графика и приблизительных значений ординат точек пересечения графиков (п. 3), достаточно взять $a = 10$. Таким образом, мы можем рассмотреть сумму площадей 6 прямоугольников со сторонами ε_1 и a (их шесть, поскольку учитываются 3 интеграла). Тогда общая ошибка не будет превосходить

$$60\varepsilon_1 + 3\varepsilon_2.$$

Возьмем $\varepsilon_1 = \varepsilon_2 = 10^{-5}$. Подставив эти значения в полученную выше оценку, мы получаем $63 \cdot 10^{-5} < 10^{-3}$, следовательно подобранные значения обеспечат требуемую точность.

3 Результаты экспериментов

Результаты эксперимента представлены в таблице.

Кривые	x	y
1 и 2	3.847760	5.308656
1 и 3	0.854103	3.512462
2 и 3	3.243928	0.924805

Таблица 1: Координаты точек пересечения

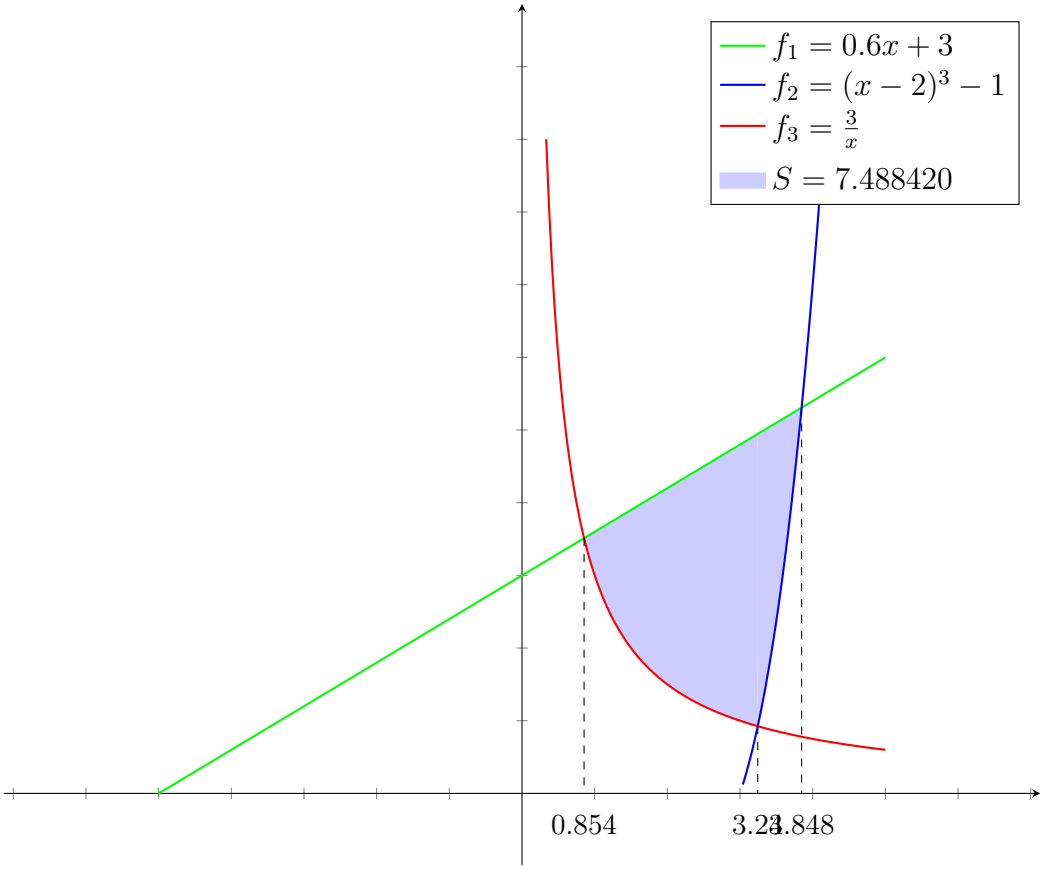

Рис. 2: Плоская фигура, ограниченная графиками заданных уравнений

4 Структура программы и спецификация функций

Разбиение программы на модули и функции, и связи между ними приведены на рис. 3.

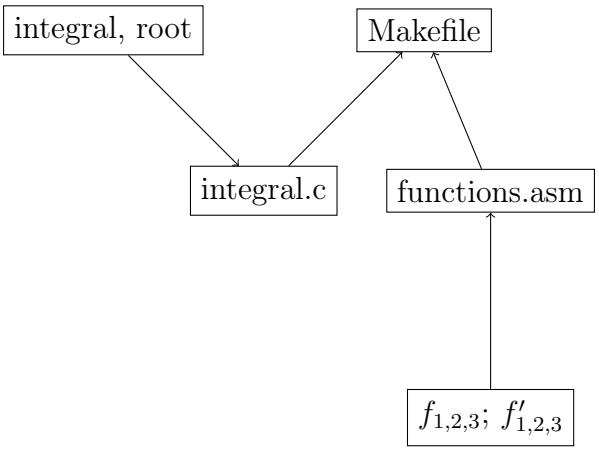

Рис. 3: Разбиение программы на компоненты

Файлы и используемые в них функции:

```
integral.c:

g2(double x), g3(double x)  - возвращают значения g2 и g3
dg1(double x), dg2(double x), dg3(double x)
                            - возвращают значения производных от g1, g2 и g3
root(double f(double), double g(double), double df(double),
double dg(double), double a, double b, double eps1)
                            - возвращает корень f(x)-g(x)=0
integral(double f(double), double a,double b, double eps2)
                            - возвращает интеграл, с заданной точностью
help(void)                  - выводит информацию о программе
main(int argc, char **argv) - считает необходимые корни и интегралы,
                              выводит ответ к задаче,
                              разбирает опции командной строки
```

```
functions.asm:

f1, f2, f3, g1      - возвращают f1(x), f2(x), f3(x), g1(x)
df1, df2, df3       - возвращают производные от f1(x), f2(x), f3(x)
```

Программой поддерживаются следующие опции командной строки (и их короткие вариации):

- `--root, -r` - вывод абсцисс точек пересечения данных функций;
- `--iterations, -i` - вывод количества итераций, потребовавшегося для вычисления корней уравнения;
- `--help, -h` - вывод информации о программе;
- `--test-root, -R` - опция, принимающая аргумент вида `F1:F2:A:B:E:R` и тестирующая функцию *root* с помощью данных из этого аргумента;
- `--test-integral, -I` - опция, принимающая аргумент вида `F:A:B:E:R` и тестирующая функцию *integral* с помощью даннымх из этого аргумента.

Последние две опции выводят результат работы тестируемого метода, абсолютную и относительную погрешности. Программа, запущенная без опций, выводит ответ на задачу (площадь плоской фигуры).

5 Сборка программы (Make-файл)

Код Makefile (за исключением предопределённых в задании флагов):

```
all: integral

integral: integral.o functions.o
        gcc -m32 -lm integral.o functions.o -o integral

integral.o:
        gcc -m32 -c integral.c -o integral.o

functions.o:
        nasm -f elf32 functions.asm -o functions.o

test: integral
        ./$< -R 1:2:0.2984:3.9999:0.00001:0.697082
        ./$< -R 1:3:0.01:0.3308:0.00001:0.330055
        ./$< -R 2:3:0.666:7.777:0.00001:0.922758
        ./$< -I 1:4.5:5.5:0.0001:-370.752
        ./$< -I 2:2.37:7.943:0.0001:11143.2
        ./$< -I 3:5.6728:1.5684:0.0001:-0.0576641

clean:
        rm *.o integral
```

Цель *all* зависит от исполняемого файла *integral*, который, в свою очередь, зависит от объектных файлов *integral.o* и *functions.o*. Эти объектные файлы зависят от *integral.c* и *functions.asm* соответственно. Данная цель собирает исполняемый файл

Цель *test* зависит от этого же исполняемого файла и запускает его с ключами test-root и test-integral (короткие вариации). В случае отсутствия исполняемого файла цель *test* пройдёт по реквизитам *integral* и соберёт исполняемый файл, после чего запустит этот файл с указанными выше ключами

Цель *clean* очищает все промежуточные файлы.

При сборке исполняемого файла с помощью *Makefile*, сначала проводится поиск файлов с полным совпадением имён, указанных в целях, реквизитах и командах. При их наличии новый файл не будет создан, следовательно при изменении исходных файлов (на Си или Асме) стоит поочерёдно воспользоваться целями *clean* и *all* перед работой с исполняемым файлом.

Также стоит учесть необходимость использования зависимостей (реквизитов) в *integral*, т.к. каждая цель, использующая *integral*, для корректной работы должна собирать исполняемый файл *integral* в случае его отсутствия.

6 Отладка программы, тестирование функций

Функция *root* тестировалась на отдельных функциях, на произвольных отрезках, удовлетворяющих условиям применения комбинированного метода (хорд и касательных). Обоснование корректности выбранных отрезков было приведено в п. 2.1. Новые тест-функции и их производные, а также результаты работы программы и результаты, полученные с помощью сервиса WolframAlpha приведены далее.

Для поиска корней были выбраны отрезки: для $g_1(x)$ – $[0.2984, 3.9999]$, для $g_2(x)$ – $[0.01, 0.3308]$, для $g_3(x)$ – $[0.666, 7.777]$.

1. $g_1(x) = ln(x) - 15x^2 + 3.89$.
2. $g_2(x) = 13x^3 - 12x^2 + 11x - 10$.
3. $g_3(x) = \frac{1}{8x^2}$.

1. $g_1'(x) = \frac{1}{x} - 30x$.
2. $g_2'(x) = 39x^2 - 24x + 11$.
3. $g_3'(x) = -\frac{1}{4x^3}$.

Номера функций	*root*	WolframAlpha
1-2	0.697082	0.697082
1-3	0.330056	0.330056
2-3	0.922758	0.922758

Таблица 2: результаты тестирования функции *root*

Функция *integral* также тестировалась на данных функциях, также с произвольными пределами интегрирования:

Функция	Отрезок интегрирования	integral	Wolfram Alpha
$f_1(x)$	$[4.5, 5.5]$	-370.752234	-370.752
$f_2(x)$	$[2.37, 7.943]$	11143.199406	11143.2
$f_3(x)$	$[5.6728, 1.5684]$	-0.057755	-0.0576641

Таблица 3: результаты тестирования функции *integral*

Численные методы и функции были отлажены утилитой gdb, встроенной в SASM и CLion.

7 Анализ допущенных ошибок

В процессе выполнения работы были допущены следующие ошибки:

1. В *Makefile integral* не имел реквизитов, из-за чего при запуске цели *test* без предварительного запуска цели *all* программа не запускалась.

Список литературы

[1] Трифонов Н. А., Пильщиков В. Н. Задания практикума на ЭВМ (1 курс). Учебное пособие, 2-е исправленное издание. — М.: МГУ, 2001. — 32 с.

[2] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1 — Москва: Наука, 1985.